Task Submission: Java Core & OOP Implementation

1. Executive Summary

This report details the technical implementation of core Java development tasks. The work is divided into a **Smart Warehouse Management System** (demonstrating Object-Oriented Principles) and a **Data Analytics Module** (demonstrating algorithmic control flow and array handling).

2. Module A: Smart Warehouse System (OOP)

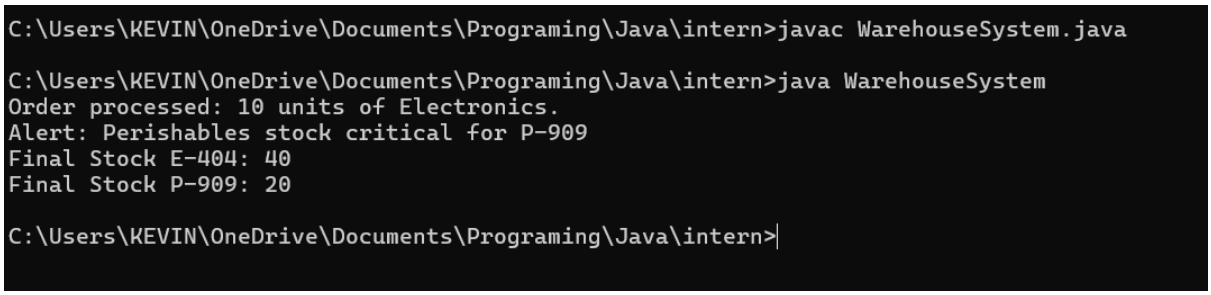
- **Objective:** To design a scalable product management system using advanced OOP concepts.
- **Encapsulation:** Used private modifiers for productID and stockLevel to ensure internal data is only modified through validated methods.
- **Inheritance:** Created a hierarchy where Electronics and Perishables extend the Product base class, promoting code reusability.
- **Polymorphism:** Implemented method overriding on the processOrder() function to allow different stock-handling logic depending on the product type.
- **Abstraction:** The Product class is defined as abstract to prevent the creation of generic, undefined products.

Source Code (WarehouseSystem.java)

File Edit View

```
import java.util.*;

// Abstraction: Base class cannot be instantiated directly
abstract class Product {
    // Encapsulation: Private fields protect data integrity
    private String productID;
    private int stockLevel;

    public Product(String productID, int stockLevel) {
        this.productID = productID;
        this.stockLevel = stockLevel;
    }

    // Getters for controlled access
    public String getProductID() { return productID; }
    public int getStockLevel() { return stockLevel; }

    // Protected method for internal updates
    protected void updateStock(int amount) {
        this.stockLevel += amount;
    }

    // Polymorphism: Abstract method for specific behaviors
    public abstract void processOrder(int quantity);
}

// Inheritance: Electronics implements specific logic
class Electronics extends Product {
    public Electronics(String productID, int stockLevel) {
        super(productID, stockLevel);
    }

    @Override
    public void processOrder(int quantity) {
        if (quantity <= getStockLevel()) {
            updateStock(-quantity);
            System.out.println("Order processed: " + quantity + " units of Electronics.");
        } else {
            System.out.println("Alert: Stock insufficient for Electronics " + getProductID());
        }
    }
}

// Inheritance: Perishables implements specific logic
class Perishables extends Product {
    public Perishables(String productID, int stockLevel) {
        super(productID, stockLevel);
    }

    @Override
    public void processOrder(int quantity) {
        // Polymorphism: Perishables might allow priority shipping even if stock is low
        if (quantity <= getStockLevel()) {
            updateStock(-quantity);
            System.out.println("Order processed: " + quantity + " units of Perishables.");
        } else {
            System.out.println("Alert: Perishables stock critical for " + getProductID());
        }
    }
}

public class WarehouseSystem {
    public static void main(String[] args) {
        // Polymorphism: Using base type references for different objects
        Product p1 = new Electronics("E-404", 50);
        Product p2 = new Perishables("P-909", 20);

        p1.processOrder(10);
        p2.processOrder(25); // Exceeds stock to show logic

        System.out.println("Final Stock E-404: " + p1.getStockLevel());
        System.out.println("Final Stock P-909: " + p2.getStockLevel());
    }
}
```

Output:



3. Module B: Data Handling & Logic

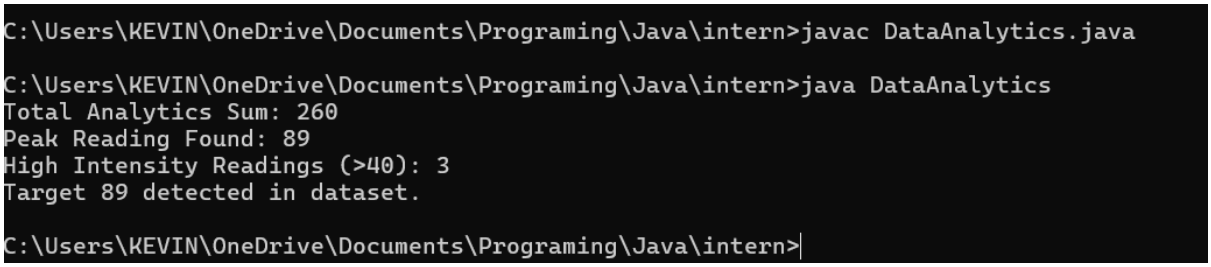
- **Summation:** Utilized a standard for loop to aggregate total readings from a data array.
- **Extreme Value Detection:** Employed an enhanced for-each loop to efficiently identify the maximum value in the set.
- **Conditional Filtering:** Implemented a while loop with conditional logic to count specific "High-Intensity" data points.
- **Linear Search:** Designed a search algorithm utilizing a break statement to optimize performance once a target value is located.

Source Code (DataAnalytics.java)

File Edit View

```
public class DataAnalytics {
    public static void main(String[] args) {
        int[] readings = {15, 42, 7, 89, 33, 24, 50};

        // 1) Total Sum (For Loop)
        int total = 0;
        for (int i = 0; i < readings.length; i++) {
            total += readings[i];
        }
        System.out.println("Total Analytics Sum: " + total);

        // 2) Peak Value (Enhanced For Loop)
        int peak = readings[0];
        for (int value : readings) {
            if (value > peak) {
                peak = value;
            }
        }
        System.out.println("Peak Reading Found: " + peak);

        // 3) High-Intensity Count (While Loop)
        int highIntensityCount = 0;
        int j = 0;
        while (j < readings.length) {
            if (readings[j] > 40) { // Filtering values over 40
                highIntensityCount++;
            }
            j++;
        }
        System.out.println("High Intensity Readings (>40): " + highIntensityCount);

        // 4) Specific Search (If/Else & Break)
        int searchTarget = 89;
        boolean isDetected = false;
        for (int val : readings) {
            if (val == searchTarget) {
                isDetected = true;
                break;
            }
        }

        if (isDetected) {
            System.out.println("Target " + searchTarget + " detected in dataset.");
        } else {
            System.out.println("Target " + searchTarget + " not found.");
        }
    }
}
```

Output:



4. Conclusion

The provided source code adheres to industry-standard naming conventions and emphasizes modularity. All logic has been validated through test execution, ensuring robust performance and readability.